Тестовое задание: Разработка QUIC-aware балансировщика нагрузки на базе Open vSwitch и OS-Ken

Цель

Разработать прототип балансировщика нагрузки для QUIC-соединений, использующий Open vSwitch (OVS) и программно-определяемый сетевой контроллер OS-Ken.

Балансировщик должен распределять новые QUIC-соединения между несколькими серверами, используя алгоритм Round Robin, и обеспечивать сохранение привязки существующих соединений к выбранному серверу при изменении IP-адреса или UDP-порта клиента.

Для идентификации сервера необходимо реализовать механизм маршрутизации на основе **QUIC Connection ID (CID)**. Серверы должны генерировать идентификаторы соединений в согласованном формате, позволяющем балансировщику определить целевой сервер непосредственно по значению CID.

Управление коммутацией должно осуществляться посредством OpenFlow. Контроллер должен устанавливать правила в OVS, обеспечивающие передачу пакетов непосредственно между клиентами и серверами без постоянного участия контроллера.

В рамках задания необходимо изучить особенности QUIC, разработать собственный механизм маршрутизации по CID, реализовать его интеграцию с OpenFlow и исследовать ограничения такого подхода.

Разрабатывать собственную реализацию протокола QUIC не требуется. Допускается использование существующих библиотек.

Инфраструктура для тестирования

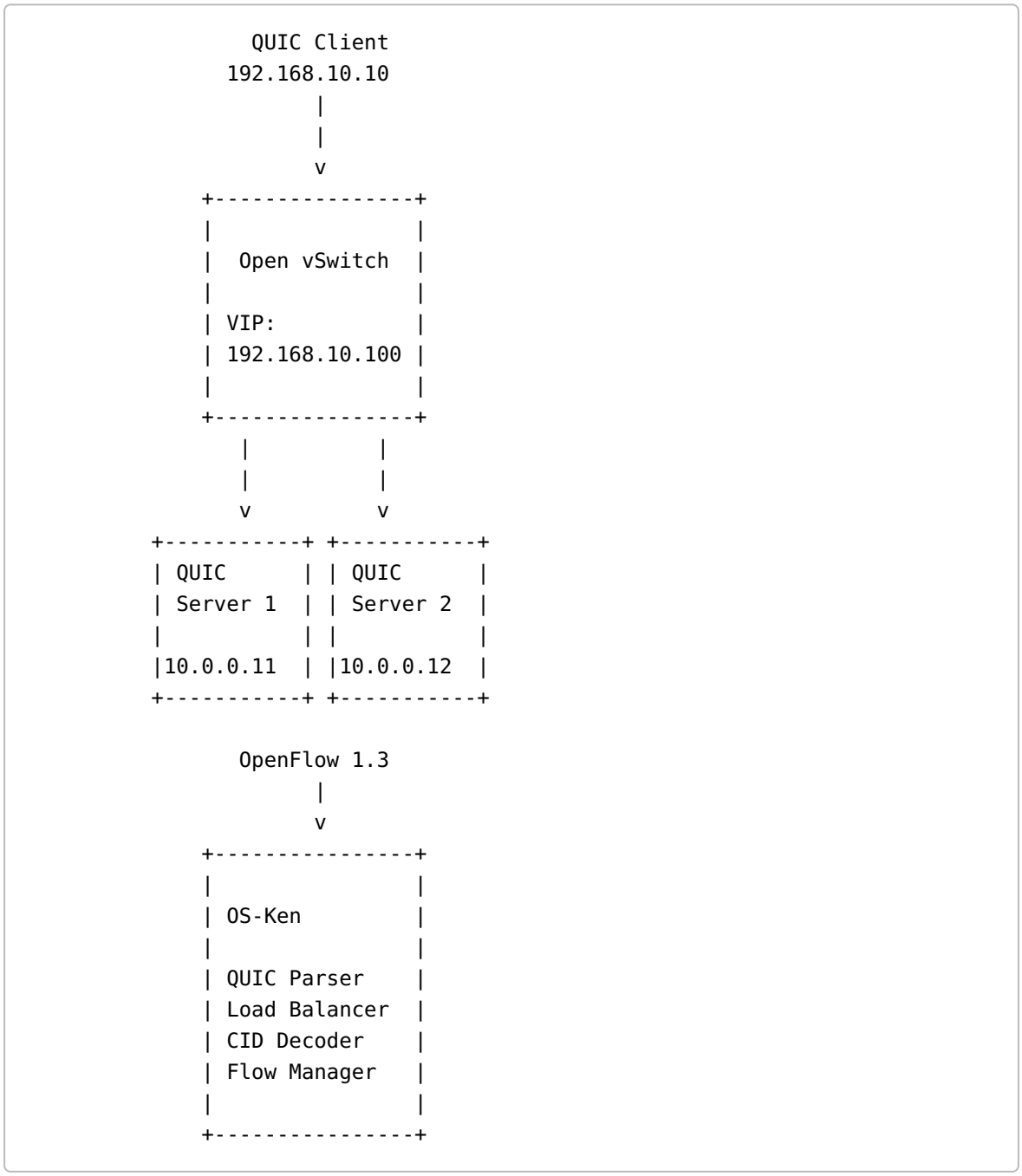
Необходимо развернуть виртуальную среду на Linux (например, Ubuntu 22.04/24.04) со следующими компонентами:

- Open vSwitch — программный коммутатор.
- OS-Ken — SDN-контроллер.
- Mininet — эмуляция сетевой топологии.
- Python 3.10+ — разработка управляющего приложения.
- aioquic или quic-go — реализация QUIC-клиента и серверов.

Допускается использование Linux Network Namespaces вместо Mininet.

Сетевая топология

Необходимо создать топологию, состоящую из одного OVS-коммутатора, одного или нескольких клиентов и двух QUIC-серверов.



Предполагается использование следующих адресов:

Компонент	IP-адрес	Порт
Client	192.168.10.10	Динамический
Load Balancer (VIP)	192.168.10.100	UDP 443

Компонент	IP-адрес	Порт
QUIC Server 1	10.0.0.11	UDP 443
QUIC Server 2	10.0.0.12	UDP 443

Допускается использование другой адресации.

OVS должен быть подключён к контроллеру OS-Ken по протоколу OpenFlow 1.3.

Необходимо обеспечить корректную двустороннюю передачу трафика между клиентом и серверами через OVS.

Важно: ответы серверов должны проходить через балансировщик, а клиент должен взаимодействовать исключительно с виртуальным IP-адресом.

Этап 1. Разработка и запуск QUIC-серверов

Необходимо разработать или адаптировать простой QUIC-сервер с использованием существующей реализации протокола.

В качестве основы допускается использовать примеры из библиотек aioquic или quic-go.

Требования

1. Развернуть два независимых QUIC-сервера.
2. Настроить прослушивание UDP-порта 443.
3. Реализовать простой сервис, возвращающий идентификатор сервера в ответ на запрос клиента.
4. Обеспечить поддержку нескольких последовательных запросов в рамках одного QUIC-соединения.
5. Реализовать логирование установления и завершения соединений.

Например, первый сервер должен возвращать:

```
Hello from QUIC Server 1
```

Второй:

```
Hello from QUIC Server 2
```

Клиент должен уметь устанавливать соединение и отправлять несколько запросов без повторного установления QUIC-соединения.

Ожидаемый результат

Оба QUIC-сервера доступны по своим IP-адресам и способны независимо обрабатывать QUIC-соединения.

Необходимо продемонстрировать успешное установление соединения с каждым сервером.

Этап 2. Реализация собственного формата QUIC Connection ID

Необходимо модифицировать механизм генерации Connection ID на QUIC-серверах.

Цель — обеспечить возможность определения сервера по значению Destination Connection ID без необходимости хранить информацию обо всех выданных CID в балансировщике.

Формат CID

Для тестового задания предлагается использовать фиксированную длину CID — 12 байт.

```
+------------------+------------------+
| Configuration ID |    Server ID     |
|      1 byte      |      1 byte      |
+------------------+------------------+
|                                     |
|            Random Data              |
|              10 bytes               |
|                                     |
+-------------------------------------+
```

Поля:

- `Configuration ID` — идентификатор формата CID. Для тестирования допускается фиксированное значение `0xA1`.
- `Server ID` — уникальный идентификатор сервера (1 или 2).
- `Random Data` — случайные данные для формирования уникальных CID.

Примеры:

```
Server 1:

A1 01 4B 82 91 57 30 4C 71 A8 B9 2D
   ^^
   Server ID = 1


Server 2:
```

```
A1 02 9C 17 42 F5 18 63 94 AB 77 21
   ^^
   Server ID = 2
```

Требования

1. Каждый сервер должен генерировать CID с собственным `Server ID`.
2. Генерация CID должна обеспечивать уникальность идентификаторов.
3. Первый CID, выданный сервером при установлении соединения, должен соответствовать заданному формату.
4. Если сервер генерирует дополнительные CID в процессе работы соединения, они также должны содержать его `Server ID`.
5. Изменение генерации CID не должно нарушать работу QUIC-соединений.

Для изменения CID допускается модификация исходного кода выбранной QUIC-библиотеки.

Ожидаемый результат

При установлении соединения клиент получает CID, содержащий идентификатор выбранного сервера.

Сервер должен регистрировать выдаваемые CID в журнале.

Например:

```
Server 1:
New connection established
Generated CID: A1014B829157304C71A8B92D

Server 2:
New connection established
Generated CID: A1029C1742F5186394AB7721
```

Примечание: предложенный формат является учебным упрощением QUIC-LB. Он не обеспечивает сокрытие идентификатора сервера и не должен рассматриваться как готовая реализация спецификации QUIC-LB.

Этап 3. Разработка QUIC Packet Parser

Необходимо реализовать модуль анализа QUIC-пакетов, который будет использоваться SDN-контроллером.

Модуль должен принимать UDP-пакет и извлекать доступную информацию из QUIC-заголовка.

Long Header

Необходимо поддержать разбор Long Header для QUIC версии 1, включая извлечение:

- типа пакета;
- версии QUIC;
- длины Destination Connection ID;
- значения Destination Connection ID;
- длины Source Connection ID;
- значения Source Connection ID.

Пример результата:

```
{
  "header_type": "long",
  "packet_type": "initial",
  "version": 1,
  "dcid": "aabbccdd",
  "scid": "11223344"
}
```

Short Header

Необходимо реализовать извлечение Destination Connection ID из Short Header.

В рамках задания допускается использовать заранее известную длину серверного CID — 12 байт.

Пример:

```
{
  "header_type": "short",
  "dcid": "a1014b829157304c71a8b92d"
}
```

Декодирование CID

Необходимо реализовать отдельную функцию, определяющую сервер по CID.

Например:

```
def decode_backend_id(dcid: bytes) -> int | None:
    ...
```

Функция должна:

1. Проверять длину CID.
2. Проверять идентификатор конфигурации.

3. Извлекать `Server ID`.
4. Проверять наличие сервера в конфигурации балансировщика.
5. Возвращать идентификатор сервера либо `None`.

Обработка ошибок

Модуль должен корректно обрабатывать:

- UDP-пакеты, не являющиеся QUIC;
- некорректные и обрезанные QUIC-заголовки;
- CID неподдерживаемой длины;
- неизвестные значения `Configuration ID`;
- неизвестные идентификаторы серверов.

Некорректный пакет не должен приводить к аварийному завершению контроллера.

Ожидаемый результат

Модуль способен извлекать CID из QUIC-пакетов и определять сервер по идентификаторам, сформированным на предыдущем этапе.

Для проверки необходимо написать несколько автоматических тестов, использующих реальные или заранее подготовленные QUIC-пакеты.

Этап 4. Разработка приложения балансировщика для OS-Ken

Необходимо реализовать отдельное приложение SDN-контроллера, наследующееся от:

```
os_ken.base.app_manager.OSKenApp
```

Приложение должно обеспечивать обработку OpenFlow-событий и управление балансировкой QUIC-соединений.

В качестве основы допускается использовать стандартный пример `simple_switch_13.py`.

Подключение коммутатора

При подключении OVS контроллер должен:

1. Получить информацию о коммутаторе.
2. Установить начальные OpenFlow-правила.
3. Настроить обработку UDP-трафика, направленного на VIP.
4. Обеспечить передачу неизвестных потоков на контроллер через `Packet-In`.

Обработка нового соединения

Когда клиент отправляет первый QUIC Initial-пакет, контроллер должен выбрать сервер с использованием алгоритма Round Robin.

Пример:

```
Connection 1 -> Server 1
Connection 2 -> Server 2
Connection 3 -> Server 1
Connection 4 -> Server 2
```

Важно учитывать, что **первый Destination CID формируется клиентом**, поэтому он не содержит достоверной информации о выбранном сервере.

При обработке нового Initial-пакета решение о выборе сервера должно приниматься балансировщиком.

Необходимо сохранять состояние первоначального обмена, чтобы повторные Initial-пакеты одного соединения направлялись на тот же сервер.

Обработка существующего соединения

После того как клиент начинает использовать CID, выданный сервером, контроллер должен определять backend на основе его значения.

Например:

```
Packet received

DCID: A1029C1742F5186394AB7721

Configuration ID = 0xA1
Server ID = 2

Selected backend: 10.0.0.12
```

Для корректной работы необходимо отличать первоначальные клиентские CID от серверных маршрутизируемых CID.

Неизвестные CID

Необходимо предусмотреть обработку пакетов, содержащих неизвестные или некорректные CID.

Контроллер не должен произвольно перенаправлять такие пакеты существующих соединений на следующий сервер по Round Robin.

Допускается отбрасывать такие пакеты с регистрацией причины.

Ожидаемый результат

Контроллер:

- обрабатывает QUIC Initial-пакеты;
- распределяет новые соединения между серверами;
- определяет сервер по маршрутизируемым CID;
- корректно обрабатывает неизвестные CID;
- регистрирует принимаемые решения в журнале.

Этап 5. Установка OpenFlow-правил

После выбора сервера контроллер должен устанавливать OpenFlow-правила, обеспечивающие прямую передачу UDP-трафика через OVS.

Требования

Для каждого нового UDP-потока необходимо:

1. Определить IP-адрес и UDP-порт клиента.
2. Определить выбранный backend.
3. Установить правило передачи клиентского трафика на выбранный сервер.
4. Установить обратное правило для передачи ответов сервера клиенту.
5. Выполнить необходимую трансляцию IP- и MAC-адресов.
6. Настроить время жизни правил.
7. Обеспечить корректную обработку первого пакета, вызвавшего `Packet-In`.

Для входящего трафика правила должны учитывать:

```
eth_type
ip_proto
ipv4_src
ipv4_dst
udp_src
udp_dst
```

Пример:

```
Match:
    ipv4_src = 192.168.10.10
    ipv4_dst = 192.168.10.100
    udp_src  = 51000
    udp_dst  = 443

Actions:
    Set destination IP = 10.0.0.11
```

```
    Set destination MAC = Server 1 MAC
    Output = Server 1 port
```

Для обратного трафика должна выполняться необходимая трансляция адресов, чтобы клиент продолжал взаимодействовать с VIP.

Требования к производительности

После установки OpenFlow-правил последующие пакеты соответствующего UDP-потока должны обрабатываться непосредственно OVS.

Передавать каждый пакет установленного UDP-потока контроллеру не требуется.

Время жизни правил должно быть настраиваемым.

Для диагностики рекомендуется использовать:

```
ovs-ofctl -O OpenFlow13 dump-flows br-int
```

Имя моста может отличаться.

Ожидаемый результат

После установления QUIC-соединения:

- UDP-пакеты передаются через OVS;
- серверы получают исходящий от клиента трафик;
- клиент получает ответы от VIP;
- в OVS присутствуют соответствующие OpenFlow-правила;
- контроллер не обрабатывает каждый пакет существующего UDP-потока.

Этап 6. Поддержка изменения адреса клиента (NAT Rebinding)

Одной из особенностей QUIC является возможность продолжить существующее соединение при изменении сетевого адреса клиента.

Необходимо исследовать и реализовать обработку такой ситуации на уровне балансировщика.

Исходное состояние

Клиент установил соединение с Server 1.

```
Client:
    IP: 192.168.10.10
    UDP port: 51000
```

```
VIP:
    IP: 192.168.10.100
    UDP port: 443

QUIC:
    DCID: A1014B829157304C71A8B92D

Backend:
    Server 1
```

Контроллер установил OpenFlow-правила для данного UDP-потока.

Изменение адреса клиента

В процессе работы соединения изменяется UDP-порт либо IP-адрес клиента.

Например:

```
Client:
    IP: 192.168.10.10
    UDP port: 52000

QUIC:
    DCID: A1014B829157304C71A8B92D
```

В результате существующие правила OVS больше не соответствуют новому UDP-потоку.

Первый пакет с новыми сетевыми параметрами должен поступить на контроллер.

Контроллер должен:

1. Получить пакет через `Packet-In`.
2. Извлечь Destination Connection ID.
3. Определить `Server ID` по CID.
4. Найти соответствующий backend.
5. Установить новые OpenFlow-правила.
6. Обеспечить передачу пакета исходному серверу.

При этом **повторно выполнять Round Robin не требуется**, поскольку пакет содержит маршрутизируемый CID.

Ожидаемый результат

После изменения UDP-порта или IP-адреса клиента соединение продолжает работать с исходным сервером.

Необходимо продемонстрировать, что данные успешно передаются после изменения сетевых параметров.

Изменение адреса должно происходить без создания нового QUIC-соединения. Для проверки необходимо использовать клиента, сохраняющего состояние существующего QUIC-соединения.

Этап 7. Исследование смены Connection ID

QUIC позволяет серверу выдавать клиенту новые Connection ID в течение одного соединения.

Новые CID могут передаваться с помощью кадров `NEW_CONNECTION_ID`.

Необходимо исследовать, как изменение CID влияет на работу разработанного балансировщика.

Требования

1. Изучить механизм выдачи дополнительных CID в QUIC.
2. Убедиться, что все CID, генерируемые одним сервером, содержат одинаковый `Server ID`.
3. Реализовать тестирование декодирования нескольких разных CID одного сервера.
4. Объяснить, почему смена CID не должна приводить к изменению выбранного backend.

Например:

```
Original CID:
    A1014B829157304C71A8B92D

New CID:
    A1019811AA75E3F0A221D47B
```

Оба CID должны маршрутизироваться на Server 1.

В отличие от балансировщика с таблицей известных CID, разработанный механизм не должен требовать регистрации каждого нового идентификатора соединения в контроллере.

Дополнительное задание

Реализовать сценарий, в котором клиент одновременно:

- изменяет IP-адрес или UDP-порт;
- переключается на другой CID, ранее выданный сервером;
- продолжает работу существующего QUIC-соединения.

Необходимо продемонстрировать, что балансировщик корректно определяет исходный сервер по новому CID.

Ожидаемый результат

Балансировщик корректно декодирует несколько CID одного сервера без хранения таблицы соответствий между отдельными CID и соединениями.

При выполнении дополнительного задания необходимо продемонстрировать сохранение работоспособности соединения после одновременной смены сетевого адреса и CID.

Этап 8. Тестирование и анализ ограничений

Необходимо разработать набор тестов и подготовить краткое описание полученных результатов.

Тест 1. Установление QUIC-соединения

Клиент устанавливает соединение с VIP.

Проверяется:

- получение Initial-пакета контроллером;
- выбор backend;
- установка OpenFlow-правил;
- успешное установление QUIC-соединения.

Тест 2. Балансировка новых соединений

Клиент последовательно устанавливает не менее 10 независимых QUIC-соединений с использованием разных UDP-потоков.

Необходимо продемонстрировать распределение соединений между серверами по алгоритму Round Robin.

Ожидаемый результат:

```
Server 1: 5 connections
Server 2: 5 connections
```

Тест 3. Сохранение привязки соединения

Клиент выполняет несколько запросов в рамках одного QUIC-соединения.

Все запросы должны обрабатываться одним сервером.

Тест 4. Изменение UDP-порта

В процессе работы QUIC-соединения изменяется исходный UDP-порт клиента.

Необходимо подтвердить, что новое OpenFlow-правило направляет трафик на исходный сервер.

Тест 5. Маршрутизация по CID

Необходимо продемонстрировать, что разные CID с одинаковым `Server ID` декодируются в один и тот же backend.

Тест 6. Обработка некорректных пакетов

Необходимо проверить поведение контроллера при получении:

- некорректного QUIC-заголовка;
- CID неподдерживаемого формата;
- неизвестного `Server ID`;
- обычного UDP-пакета, не являющегося QUIC.

Контроллер не должен аварийно завершаться.

Тест 7. Передача трафика через OVS

Необходимо подтвердить, что после установки OpenFlow-правил трафик существующего UDP-потока не требует обработки каждого пакета контроллером.

Для проверки рекомендуется использовать статистику OpenFlow-правил и количество сообщений `Packet-In`.

Анализ ограничений

В README.md необходимо отдельно ответить на следующие вопросы:

1. Почему первый QUIC Initial нельзя маршрутизировать исключительно по серверному CID?
2. Чем CID-based маршрутизация отличается от балансировки UDP по 5-tuple?
3. Почему Short Header сложнее анализировать, чем Long Header?
4. Почему при смене CID балансировщик с таблицей ранее обнаруженных идентификаторов может потерять привязку соединения?
5. Почему разработанный механизм способен определить сервер по новому CID без хранения состояния каждого соединения?
6. Какие ограничения накладывает использование обычных OpenFlow-правил, сопоставляющих пакеты по 5-tuple?
7. Что произойдёт, если один UDP-поток будет использоваться для нескольких QUIC-соединений с различными CID?
8. Что произойдёт с существующим QUIC-соединением при отказе выбранного сервера?
9. Какие недостатки безопасности и конфиденциальности имеет предложенный формат CID?
10. Какие изменения потребуются для реализации полноценного QUIC-LB без обращения к контроллеру для каждого нового UDP-потока?

Важно: стандартный OpenFlow не предоставляет возможности сопоставлять пакеты непосредственно по QUIC Connection ID. Поэтому в рамках задания допускается установка правил по 5-tuple после определения backend контроллером.

Необходимо понимать и документировать ограничения такого подхода. В частности, установленное правило по 5-tuple не различает QUIC-соединения с разными CID, использующие один UDP-поток.

Детали реализации

1. Приложение OS-Ken

Необходимо создать класс, наследующийся от:

```
os_ken.base.app_manager.OSKenApp
```

Рекомендуется использовать следующие события:

- `EventOFPSwitchFeatures` — настройка коммутатора при подключении.
- `EventOFPPacketIn` — получение пакетов, требующих решения контроллера.
- `EventOFPStateChange` — отслеживание подключения и отключения коммутатора.
- `EventOFPFlowRemoved` — обработка уведомлений об удалении правил, если соответствующие уведомления запрошены.
- `EventOFPErrorMsg` — обработка ошибок OpenFlow.

Приложение должно содержать отдельные логические компоненты:

- QUIC Packet Parser;
- CID Decoder;
- Backend Selector;
- OpenFlow Flow Manager.

Допускается другая архитектура при условии логического разделения функций.

2. Управление состоянием

Контроллер должен хранить минимальный объём информации, необходимой для обработки начального обмена и установленных UDP-потоков.

Рекомендуется предусмотреть:

- соответствие начальных UDP-потоков выбранным серверам;
- состояние первоначального QUIC-обмена;
- список доступных backend-серверов;
- информацию об установленных OpenFlow-правилах.

Хранить все выданные серверами CID не требуется.

3. Управление OpenFlow-правилами

Правила должны иметь настраиваемые значения `idle_timeout`.

Необходимо предусмотреть возможность удаления устаревших правил.

Желательно использовать OpenFlow `cookie` для идентификации правил, созданных балансировщиком.

4. Обработка ошибок

Контроллер должен корректно обрабатывать:

- потерю соединения с OVS;
- получение некорректных пакетов;
- отсутствие backend в конфигурации;
- ошибки установки OpenFlow-правил;
- получение пакетов с неизвестными CID.

Ошибки должны регистрироваться в журнале.

Ожидаемый результат

Результат работы должен быть представлен в виде публичного Git-репозитория с историей разработки.

Репозиторий должен содержать:

- Исходный код приложения OS-Ken.
- Модуль разбора QUIC-пакетов.
- Реализацию декодирования CID.
- QUIC-клиент и QUIC-серверы.
- Реализацию генерации маршрутизируемых CID на серверах.
- Скрипты создания и удаления тестовой топологии.
- Автоматические тесты.
- Файл `requirements.txt` или эквивалентное описание зависимостей.
- README.md с описанием архитектуры и инструкцией по запуску.

README.md

Документация должна содержать:

1. Описание реализованного функционала.
2. Схему сетевой топологии.
3. Инструкцию по установке зависимостей.
4. Команды для запуска контроллера, OVS и серверов.
5. Инструкцию по запуску QUIC-клиента.
6. Описание используемого формата CID.
7. Примеры OpenFlow-правил.
8. Результаты тестирования.
9. Описание ограничений реализации.

Демонстрация

Необходимо предоставить видео или воспроизводимое текстовое описание с журналами, подтверждающее:

- установление QUIC-соединений;
- распределение новых соединений между серверами;

- генерацию CID с идентификаторами серверов;
- установку OpenFlow-правил;
- передачу пакетов непосредственно через OVS;
- сохранение привязки соединения после изменения UDP-порта;
- корректное декодирование нескольких CID одного сервера.

Приветствуется автоматизация развёртывания стенда и запуск тестов одной командой.

Стек технологий

Компонент	Технологии
SDN-контроллер	OS-Ken
Протокол управления	OpenFlow 1.3
Программный коммутатор	Open vSwitch
Эмуляция сети	Mininet или Linux Network Namespaces
Язык контроллера	Python 3.10+
Реализация QUIC	aioquic или quic-go
Диагностика	tcpdump, Wireshark, ovs-ofctl
Тестирование	pytest или аналог
Инструменты	git, pip, make, curl

Система оценивания (максимум 100 баллов)

Критерий	Баллы
Развёртывание сетевой топологии и QUIC-серверов	10
Генерация маршрутизируемых CID на серверах	15
Реализация QUIC Parser и CID Decoder	15
Интеграция балансировщика с OS-Ken	15
Корректная установка OpenFlow-правил и двусторонняя передача трафика	15
Реализация Round Robin для новых соединений	10
Сохранение привязки соединений при NAT rebinding	10
Тестирование, документация и анализ ограничений	10
Итого	**100**

Для успешного прохождения необходимо набрать не менее **70 баллов**.

Дополнительные задания

За дополнительные задания можно получить до 20 баллов сверх основной оценки:

Задание	Баллы
Демонстрация успешной миграции с одновременной сменой IP/UDP-порта и CID	+10
Реализация дополнительных механизмов диагностики, мониторинга состояния балансировщика и очистки устаревших правил	+5
Исследование возможности обработки CID непосредственно в datapath без участия контроллера	+5

Примечания

1. Полноценная реализация спецификации QUIC-LB не требуется. Достаточно согласованного формата CID с идентификатором сервера.

2. Не требуется самостоятельно реализовывать QUIC или криптографические механизмы TLS 1.3.

3. Допускается модификация существующей QUIC-библиотеки для изменения механизма генерации CID.

4. Не требуется обеспечивать прозрачное переключение существующего QUIC-соединения между серверами при отказе одного из них.

5. Поддержка нескольких независимых QUIC-соединений внутри одного UDP-потока не является обязательной, но ограничения такого сценария необходимо описать.

6. Использование `Packet-In` для обработки новых UDP-потоков допускается. Передача каждого пакета существующего потока контроллеру не требуется.

7. Приветствуется применение собственных решений, не предусмотренных описанной архитектурой, если они позволяют выполнить требования задания.

8. При выполнении задания рекомендуется руководствоваться следующими документами:

9. RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport.

10. RFC 9312 — Manageability of the QUIC Transport Protocol.
11. QUIC-LB: Generating Routable QUIC Connection IDs.
12. Документация OS-Ken.
13. Документация Open vSwitch.

Основное внимание при проверке будет уделяться пониманию сетевых протоколов, корректности взаимодействия компонентов и обоснованности выбранных технических решений.